CODA: Reliable False-Alarm Control for Quantized Anomaly Detectors on Heterogeneous Microcontrollers

Abdelaziz Saud
PhD Program: Computer Science
University of Sharjah
Student ID: U25200513 Email: U25200513@sharjah.ac.ae
Course 1501830, Fall 2026/2027 Instructor: Dr. Hamza Ouarnoughi

***Abstract*—Anomaly detectors for cyber-physical systems are calibrated at full precision and then quantized to run on microcontrollers. Quantization shifts the anomaly-score distribution while largely preserving its ranking, so a conformal threshold inherited from the full-precision model can lose its false-alarm guarantee while accuracy metrics such as AUC show no change. We propose CODA (Calibrate Once, Deploy Anywhere), which calibrates the threshold once on a host GPU and gives each device a label-free correction measured from how its integer arithmetic perturbs the score. We will test whether this keeps the realized false-alarm rate within a target bound on three microcontroller families (STM32, ESP32, RP2350) using public industrial anomaly-detection benchmarks, and report latency and memory overhead on each board.**

***Index Terms*—conformal prediction, anomaly detection, quantization, TinyML, cyber-physical systems**

I. Problem Statement and Motivation

Industrial sensors increasingly detect faults on the device itself, which cuts latency and keeps raw data local. To fit in kilobytes of SRAM, models are converted to 8-bit integers (INT8) after training [1]–[3]. Operators also need a predictable false-alarm rate. Conformal anomaly detection sets a threshold that bounds false alarms on normal data at a chosen level α [4], [5]. This bound holds only for the model used in calibration.

In practice, the threshold is set once at full precision and copied to every device. Each device runs a different integer model, with its own kernels and rounding, so its scores drift and the guarantee is lost. Recalibrating each device is often impractical. The calibration set may not fit in flash, and its owner may not be allowed to copy it to every device in a fleet.

Research question: Can a threshold calibrated once at full precision, plus a label-free per-device shift correction, keep the realized false-alarm rate of INT8 anomaly detectors on heterogeneous microcontrollers within a target bound?

Hypothesis: On all three boards, the corrected threshold keeps the false-alarm rate on normal test data within $\alpha + 0.01$ (for $\alpha = 0.01$ and 0.05), while the inherited threshold exceeds it, and detection recall stays within 5 percentage points of per-device recalibration.

TABLE I
Closest work and what it leaves open

Work	What it shows	What is missing
Bysani [10]	Inherited threshold fails after quantization	Guarantee; real hardware
Tong et al. [11]	Conformal on compressed LLMs	Reuse of one threshold
Zhou et al. [12]	Certifies FP/INT agreement	False-alarm control
TCUQ [13], HDUQ [14]	Conformal on MCUs	Transfer from full precision

II. Related Work and Research Gap

Table I summarizes the closest work. A NeurIPS 2026 workshop study showed that 4-bit quantization of robot-policy failure detectors leaves AUC almost unchanged while the coverage of an inherited conformal threshold drops from 0.77 to 0.41 (target 0.90), because quantization roughly translates the score distribution [10]; its repair is empirical and its quantization simulated. A 2026 benchmark measured conformal prediction on compressed LLMs, but recalibrates every compressed model [11]. A retrieval study gives a conformal certificate that quantized and full-precision outputs agree [12]; it does not address false alarms. On microcontrollers, TCUQ and HDUQ-HAR run conformal calibration on the deployed model directly [13], [14]. INT8 kernel paths on different hardware also diverge layer by layer [15], which suggests that each chip needs its own correction. Conformal theory for non-exchangeable data exists [6], [7] but has not been applied to this setting.

To our knowledge, no existing method restores a conformal false-alarm guarantee for a full-precision threshold deployed on integer hardware, with proof, on real microcontrollers, for streaming sensor data.

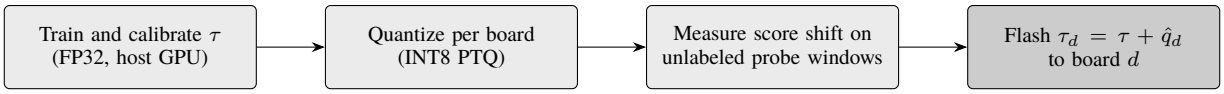

Fig. 1. The CODA pipeline. Calibration data stays on the host; each board receives one number, τ_d.

III. Proposed Method

A. Problem Formulation

Let $s(x)$ be the anomaly score of window x under the full-precision model, and $s_d(x)$ its score on device d. On the host, standard split-conformal calibration on normal data gives a threshold τ with false-alarm rate at most α [4]. On each device we measure the score shift $s_d(x) - s(x)$ on a set of unlabeled probe windows and set

$$\tau_d = \tau + \hat{q}_d, \tag{1}$$

where $\hat{q}_d$ is a high conformal quantile (level $1 - \delta$) of that shift. The false-alarm rate on device d is then at most $\alpha + \delta$ when probe and test windows are exchangeable. We will prove this formally and extend it to time series using block calibration [6].

B. Approach

Fig. 1 shows the pipeline. (1) Windows of length 64 (stride 1) are z-normalized with training statistics. (2) A 1D-convolutional autoencoder (three conv layers, under 100 KB in INT8) is trained on normal windows; the score is reconstruction mean-squared error. The last 20% of normal training data, split in time, is the calibration set. (3) Each board's model is produced by TensorFlow Lite full-integer post-training quantization with 500 representative windows, and compiled with TFLite Micro using CMSIS-NN (STM32, RP2350) or ESP-NN (ESP32) [2]. (4) 500 probe windows are streamed to each board over UART, and the on-device scores give the shift. (5) τ_d is flashed to the board. No calibration data leaves the host.

IV. Evaluation Plan

A. Datasets and Baselines

The primary benchmark is the Server Machine Dataset (SMD) [8]. SWaT (water treatment, available on request) is the industrial case, and NASA SMAP/MSL is the backup. Common anomaly benchmarks have known labeling flaws [9], so the main metric is the false-alarm rate on normal data, where labels are least disputed.

The boards are an STM32 Nucleo (Cortex-M4/M7), an ESP32-S3 (Xtensa LX7), and a Raspberry Pi Pico 2 (RP2350, Cortex-M33), costing under 100 USD in total. Training uses existing lab GPUs.

We compare against four baselines: (B1) the inherited threshold τ; (B2) per-device recalibration with the full calibration set, as an upper bound; (B3) on-device streaming quantile recalibration, following TCUQ and adaptive conformal inference [7], [13]; and (B4) the label-free realignment repair of [10].

TABLE II
Planned experiments

ID	Question	Setup	Measure
E1	Does the inherited threshold fail?	3 boards, 3 datasets	FAR, AUC
E2	Does CODA fix it?	CODA vs. B1–B4	FAR, recall, latency, SRAM
E3	Does the shift depend on data?	3 probe sources; 50, 200, 500 probes	FAR
E4	Does it hold at lower precision?	INT8 vs. INT4; 3 model sizes	FAR, recall, memory

TABLE III
Risk register

Risk	Level	Mitigation
Shift depends on data	M	Keep a small in-domain probe set on the host
Bound too loose	M	Use a joint quantile instead of a union bound
"Why not recalibrate?"	M	Measure data size and privacy cost; report B2
SWaT access delayed	M	Start with SMD and SMAP/MSL
Board or toolchain fails	L	Two boards are enough; drop RP2350 first
Tampered probe set	L	Hash and sign the probe set
Data ethics	L	Public, non-personal data only

B. Metrics

Task metrics: realized false-alarm rate versus the target α (0.01, 0.05, and 0.1), detection recall, and AUC. Systems measures: inference latency per window (on-chip cycle counter), peak SRAM, and flash, including the overhead of the correction. All results are averaged over 5 seeds (training and calibration split), with 95% confidence intervals.

C. Ablations and Comparisons

Table II lists the planned experiments. E1 tests the failure; E2 tests the hypothesis against B1–B4; E3 and E4 are ablations on probe source and size, and on bit-width and model size.

V. Risk Analysis

The largest risk is that the score shift depends on the input data and not only on the integer arithmetic. In that case a generic probe set would give the wrong correction; experiment E3 tests this directly. Table III lists all risks with their mitigations.

VI. Timeline and Deliverables

Week 6: proposal. Weeks 7–10: data pipeline, training and calibration, quantization for three boards, baselines B1–B2; Progress Report 1 (Week 10). Weeks 11–13: proof of the bound, on-device runs, E1–E3, baselines B3–B4; Progress Report 2 (Week 13). Weeks 14–16: E4, time-series extension, 6–8 page IEEE paper, artifact, and defense (Week 16).

VII. Reproducibility and Ethics

Repository: https://github.com/as4193/CODA. It currently holds the project structure and this proposal; training, quantization, firmware, and evaluation code will be added as each milestone is completed. Seeds 0–4 will be fixed for training and data splits. We will record the dataset versions (SMD from the OmniAnomaly release, SWaT by release date) with checksums, and pin the versions of Python, PyTorch, TensorFlow/TFLite Micro, STM32CubeIDE, and ESP-IDF in an environment file and a Dockerfile. Ethical, privacy, or security notes: only public, non-personal industrial data are used; the probe-set tampering risk is analyzed in Table III.

AI Assistance Disclosure

Claude (Anthropic) was used to brainstorm project directions, search and summarize recent literature, and draft the proposal text and LaTeX, and to look up author lists for recent references. I verified the title, authors, and venue of each reference against the original paper or its arXiv/publisher page, and checked the method, hypothesis, and experiment plan against the course guidelines. I reviewed and edited the final text and take responsibility for its content.

References

[1] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF CVPR*, 2018.

[2] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. MLSys*, 2021.

[3] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Proc. NeurIPS*, 2020.

[4] S. Bates, E. Candès, L. Lei, Y. Romano, and M. Sesia, "Testing for outliers with conformal p-values," *Ann. Statist.*, vol. 51, no. 1, 2023.

[5] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Found. Trends Mach. Learn.*, vol. 16, no. 4, pp. 494–591, 2023.

[6] R. F. Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal prediction beyond exchangeability," *Ann. Statist.*, vol. 51, no. 2, 2023.

[7] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," in *Proc. NeurIPS*, 2021.

[8] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. ACM SIGKDD*, 2019.

[9] R. Wu and E. J. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, 2023.

[10] A. Bysani, "Rank-preserving, threshold-breaking: Post-training silently decalibrates inherited safety monitors in VLA policies," in *Proc. NeurIPS Workshop on Post-Training Adaptation of Robot Foundation Models (RoboPAD)*, 2026.

[11] Y. Tong, Y. Wang, Y. Wan, T. Zhang, J. Dong, and J. Yuan, "Does compression preserve uncertainty? A unified benchmark for quantized and sparse LLMs via conformal prediction," arXiv:2606.01850, 2026.

[12] L. Zhou, A. Zirilli, D. Solombrino, R. Dessì, and E. Rodolà, "The undetected damage of quantization on retrieval and how to fix it," arXiv:2609.24322, 2026.

[13] I. Lamaakal, C. Yahyati, K. El Makkaoui, I. Ouahbi, and Y. Maleh, "TCUQ: Single-pass uncertainty quantification from temporal consistency with streaming conformal calibration for TinyML," arXiv:2508.12905, 2025.

[14] I. Lamaakal, C. Yahyati, Y. Maleh, K. El Makkaoui, and I. Ouahbi, "Efficient uncertainty aware human activity recognition on microcontrollers using hyperdimensional computing and conformal prediction," *Sci. Rep.*, vol. 16, Art. no. 26262, 2026.

[15] T.-R. Chen, "The integer alibi: Localizing cross-kernel divergence in INT8-quantized LLM inference," arXiv:2608.13756, 2026.